



SW개발/HW제작 설계서

프로젝트 명 :
부족한 데이터를 보강하기 위한
제조 데이터 기반의 생성형 이미지 AI 모델 개발

2025. 10. 30

genboost

| 목차

-
1. 요구사항 정의서
 2. 서비스 구성도
 3. 서비스 흐름도
 4. 메뉴 구성도
 5. 화면 설계서
 6. **UI, UX** 정의서
 7. 엔티티 관계도
 8. 기능 처리도
 9. 알고리즘 명세서
 10. 알고리즘 상세설명서
 11. 데이터 수집처리 정의서
 12. 프로그램 목록
 13. 테이블 정의
 14. 핵심 소스코드
 15. 참조
-

| 요구사항 정의서

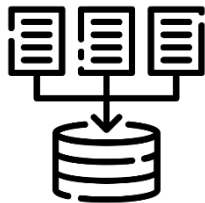
구분	기능	설명
SW	사용자의 이미지 입력	웹 브라우저(Frontend)를 통해 결함 증강을 원하는 제품의 이미지를 입력한다. 입력되는 이미지는 기반이 될 제품 이미지 외 증강을 원하는 결함이 있는 참조 이미지와 그 mask를 포함한다.
	이미지 전처리	Diffusion 모델에 입력할 때 입력 이미지의 정합성을 위해 크기 통일 및 RGB 채널을 맞추고 안정적인 학습을 위해 픽셀값을 정규화하여 Diffusion 모델에 입력하기 위한 텐서로 데이터를 변환한다.
	결함 정보 추출	입력된 결함 이미지와 결함의 mask 이미지를 통해 증강하고자 하는 결함 정보를 추출한다.
	학습용 데이터셋 제작	Real-IAD 데이터 셋을 이용해 모델 학습 데이터셋을 구축한다.
	조건 입력 생성	결함의 위치와 유형을 모델이 인식할 수 있도록 조건화된 입력을 생성한다. 기반 이미지 텐서와 결함 정보를 통합해 모델의 입력 조건을 구성한다.
	모델 구조 설계	모델을 학습시키고 튜닝하여 모델이 다양한 결함을 효과적으로 인식하여 생성해낼 수 있도록 설계한다.
	Inference 수행	입력된 조건을 반영하여 학습된 Diffusion 모델로 사용자가 입력한 제품에 요청한 결함이 바르게 증강된 이미지를 생성한다.
	이미지 평가	평가지표를 통해 생성된 이미지가 현실적이며 사용자 요구사항에 부합하는지 평가한다.
	이미지 반환	Diffusion 모델로 생성된 이미지를 사용자에게 다시 웹 브라우저(Frontend)로 반환한다.

| 서비스 구성도 - 서비스 시나리오

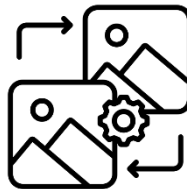
데이터 증강 서비스 시나리오



1. 사용자 입력
(Frontend)



2. 서버 처리
(Django 기반
Backend)



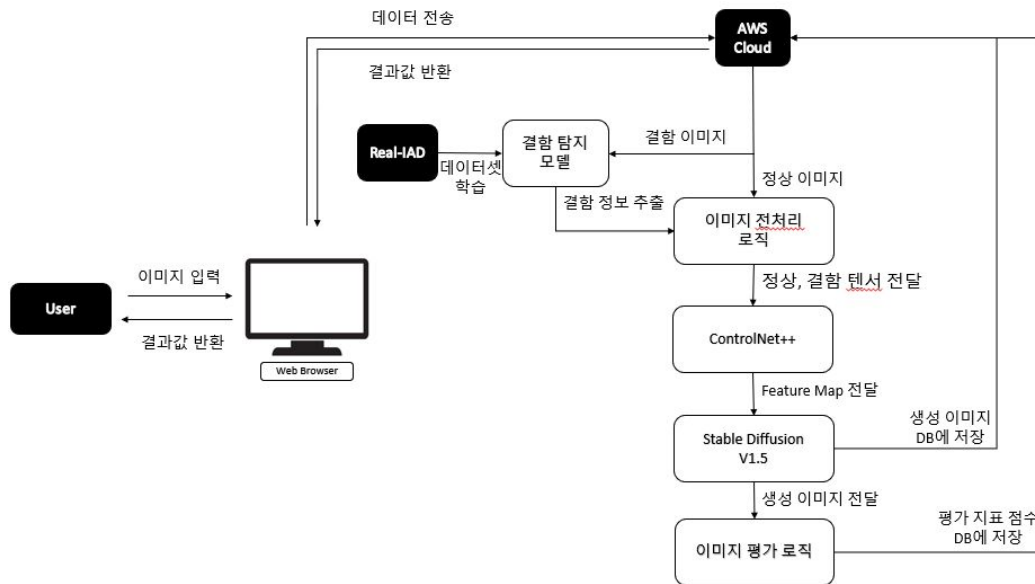
3. 결과 제공
(Frontend)

데이터 증강 서비스 시나리오

1. **사용자 입력**
- 사용자가 웹 브라우저(Frontend)를 접속하여 이미지를 업로드한다.
2. **서버처리**
- 조건에 맞게 데이터를 전처리 후 모델이 결함이 발생한 이미지를 생성한다. 생성된 이미지는 DB에 저장한다.
3. **결과제공**
- 사용자에게 모델이 결함이 발생한 제품의 이미지를 전송해준다.

| 서비스 구성도 - 서비스 시나리오

S/W 흐름도



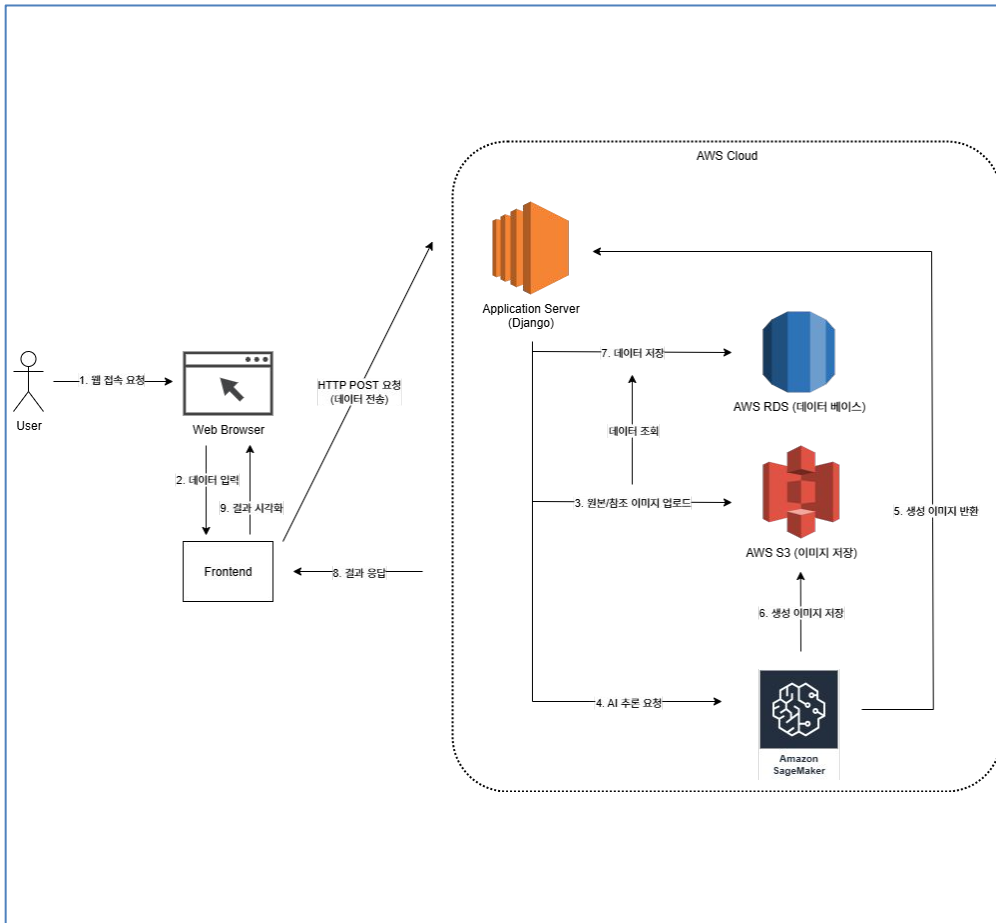
back-end

1. Django 기반 REST API 서버: 사용자 요청 처리, 유효성 검사, 모델 호출 관리
2. 파일 관리 및 데이터베이스 연동: 사용자 입력/출력 이미지와 실험 조건, 생성 결과 로그 관리 (예: PostgreSQL, SQLite)
3. 비동기 작업 처리: 모델 inference 과정에서 Celery + Redis 기반의 비동기 처리 적용 가능

front-end

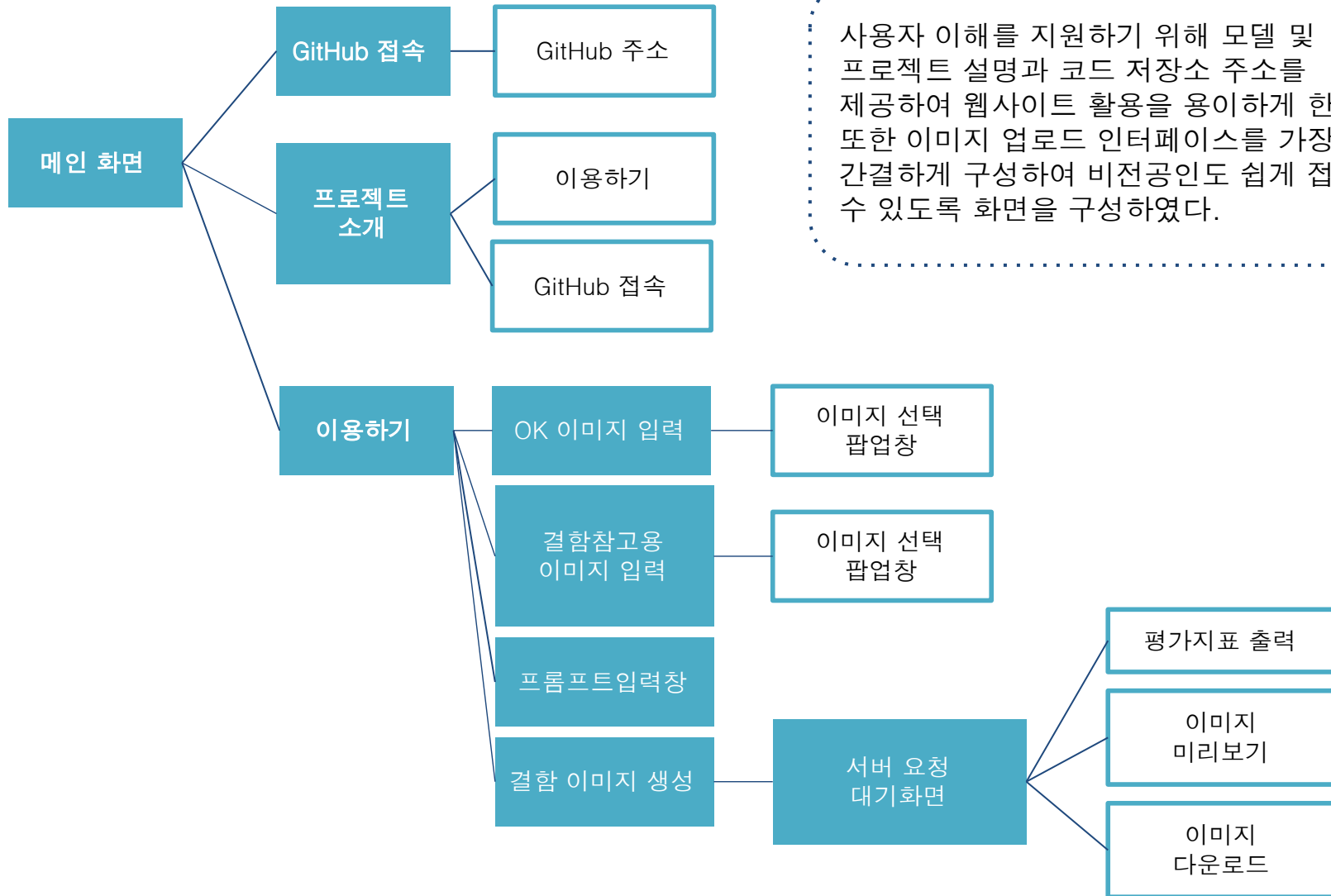
1. 웹 프레임워크: React.js 또는 Vue.js 기반 UI 개발
2. 시각화 모듈: 업로드된 이미지와 생성 결과 이미지를 직관적으로 비교할 수 있도록 Before vs After 뷰어 적용
3. 사용자 친화적 인터페이스: 정상 이미지 업로드, 결함 mask 이미지 업로드, 프롬프트 입력

| 서비스 흐름도

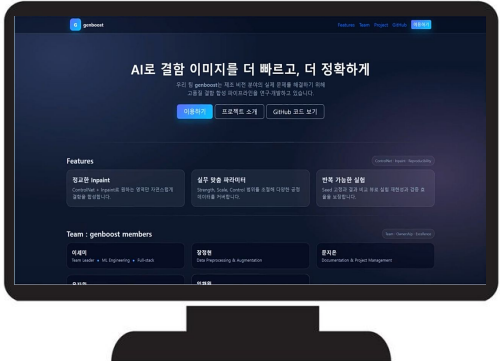
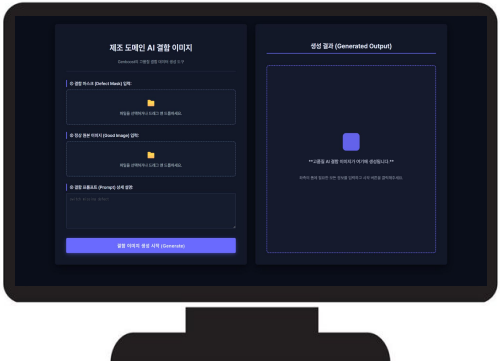


1. 웹접속 요청
: Django 기반 REST API 서버: 사용자 요청 처리, 유효성 검사, 모델 호출 관리
2. 데이터 입력
3. 원본/참조 이미지 업로드
4. AI 추론 요청
: Stable Diffusion v1.5: 사전 학습된 대규모 이미지 생성 모델 기반. 모델의 성능 및 효율성 향상을 위한 ControlNet과 LoRA를 추가적으로 사용
5. 데이터 저장
: S3 스토리지를 통한 학습 데이터 및 생성 결과 이미지 관리
6. 결과응답
: 평가 지표와 함께 제작한 생성 이미지를 반환
7. 결과 시각화
: 웹 UI에서 실험 기록과 결과를 조회 가능하게 하여 현장 적용성 강화

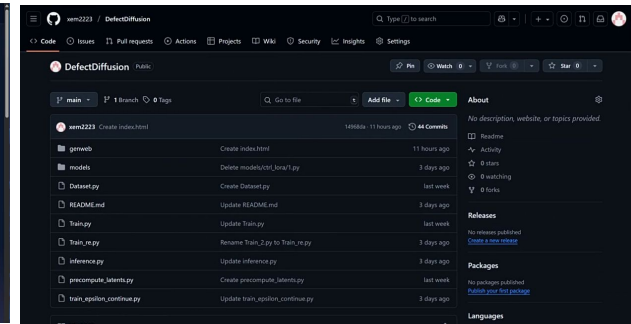
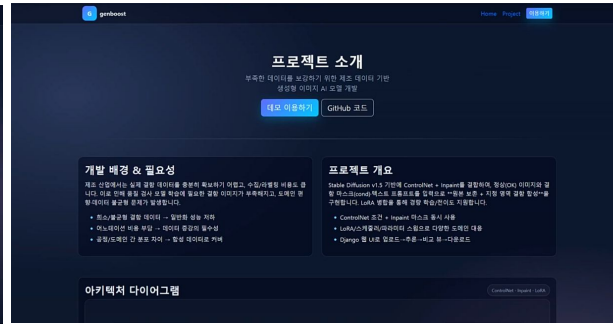
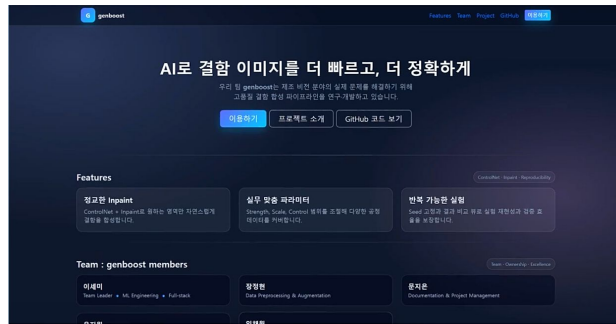
| 메뉴 구성도



| 화면 설계서

	기능 명 결함 이미지 생성 UI
기능설명 원하는 제품의 결함 이미지를 생성하는 UI이다. 목표로 하는 제품의 정상 이미지와 결함 참고용 이미지를 선택해 입력할 수 있다. 이미지 입력 후, 하단의 “이미지 생성” 버튼을 누르면 목표한 결함 이미지와 그 평가 지표가 출력된다.	처리내용 1. 정상 및 결함 참고용 이미지 입력 UI 좌측 2개의 빈 공간을 클릭하면 이미지를 선택할 수 있는 파일 탐색기 창이 열린다. 파일 탐색기에서 원하는 이미지를 선택해 입력한다. 이미지 입력 후 하단의 “이미지 생성” 버튼을 누르면 사전 Real IAD 데이터 셋으로 학습된 인공지능에 추론을 요청한다. 2. 이미지 생성 및 평가 지표 반환 버튼 클릭 후 인공지능은 주어진 이미지를 토대로 목표 제품에 원하는 결함이 증강된 이미지를 생성한다. 생성된 이미지는 측정된 평가지표와 함께 사용자에게 반환된다.
비고 해당 모델을 효과적으로 설명하기 위한 부연설명 페이지를 제공한다.	
	요구사항 명 정상 이미지 및 결함 참고용 이미지

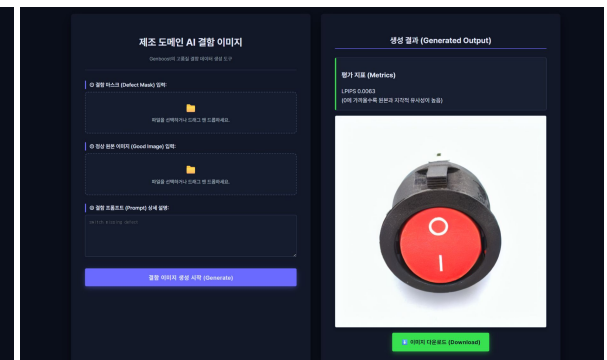
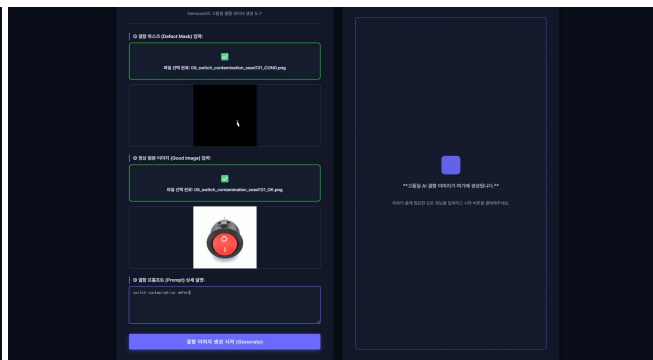
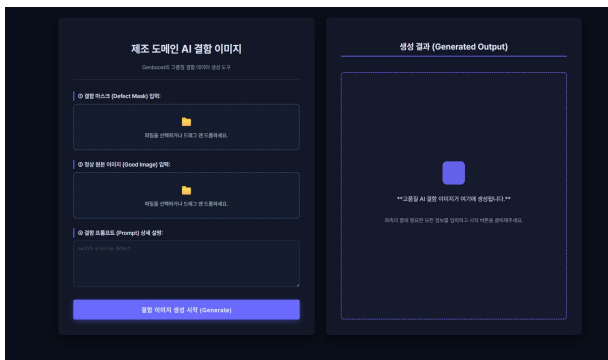
| 화면 설계서 - 사용자인터페이스 (sw)



1. 메인페이지 접속
: 전반적인 모델 소개 및
팀원 소개

1-1. 프로젝트 소개
: 프로젝트의 개발 배경
및 아키텍처 상세 설명

1-2. GitHub 코드
: 학습된 모델에 대한 코드
및 자료를 확인가능



1-3-1. 이미지 생성 페이지

1-3-2. 이미지 입력
: 프롬프트, OK이미지,
결함 MASK 이미지를 입력

1-3-3. 이미지 생성
: 입력한 이미지들과
프롬프트를 바탕으로 결함
이미지를 생성

| UI/UX 정의서 - 화면 설계서 (1)

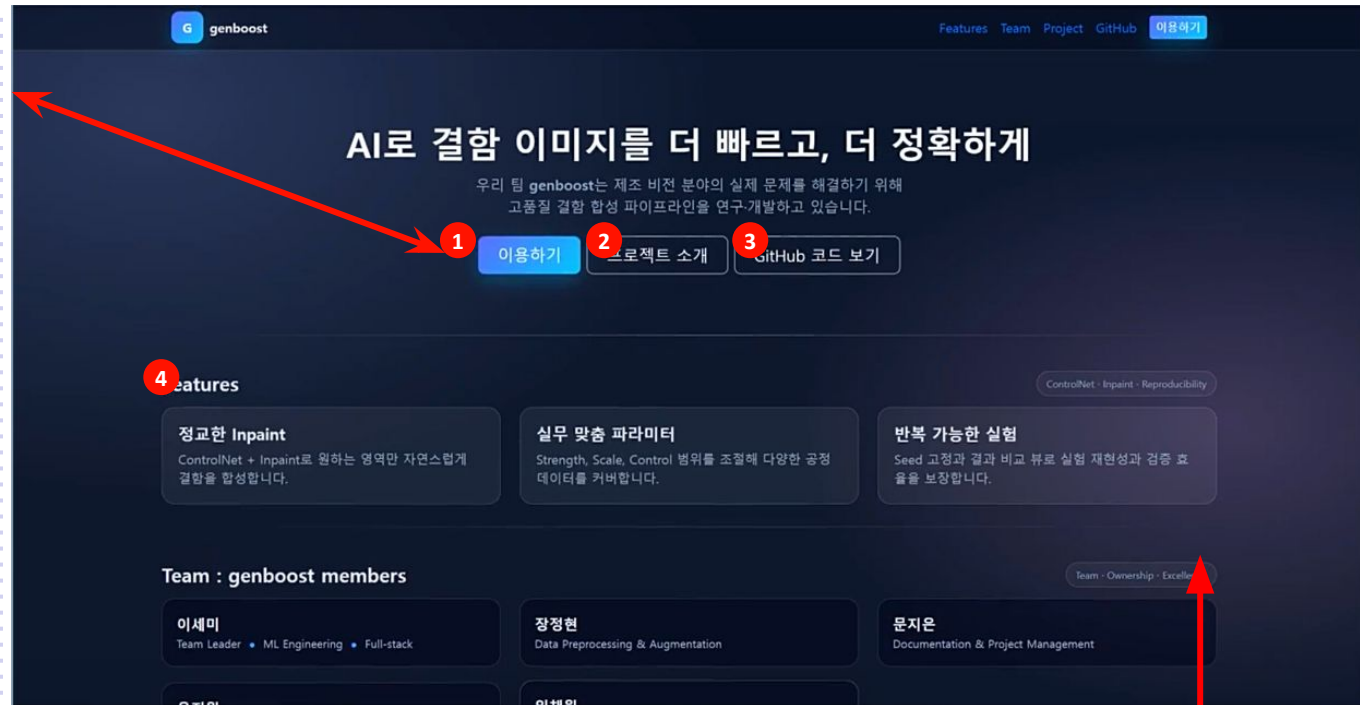
■ 메인 웹페이지

1. [이미지 생성 모델 이용하기]

가장 메인이 되는 버튼으로,
해당 버튼을 클릭 시, 학습된
모델을 바탕으로 원하는 결함
이미지를 제작할 수 있다.
(상세 화면 구성은 화면설계서
3을 참고할 것)

2. [프로젝트 소개]

프로젝트에 사용된 모델에 대한
설명 및 프로젝트의 목표성에
대해 설명한 페이지. 상세한
설명을 제공하여 사용자가
웹사이트의 효과적인 활용을
기대할 수 있게 한다.
(상세 화면 구성은 화면설계서
2를 참고할 것)



3. [GitHub 코드 방문하기]

프로젝트에서 사용된 코드를
GitHub 웹사이트를 통해 제공한다.
제공되는 데이터의 종류에는
학습코드, 추론코드, 전처리된
데이터셋, 모델 및 웹 코드 등이
있다.

4. [메인 페이지 하단 설명]

프로젝트에 참여한 팀 멤버 및
간략화 된 모델 소개가 적혀있다.
상세 사항은 2. [프로젝트 소개]를
통해 확인할 수 있다.

| UI/UX 정의서 - 화면 설계서 (2)

■ 프로젝트 및 모델 소개 페이지.

[페이지 설명]

프로젝트에 사용된 모델에 대한 설명 및 프로젝트의 목표성에 대해 설명한 페이지. 상세한 설명을 제공하여 사용자가 웹사이트의 효과적인 활용을 기대할 수 있게 한다.

모델의 상세 아키텍처 다이어그램과 모델 사용에 있어 어떤 점이 우리의 모델이 타 모델보다 이점을 갖는지를 설명한다. 해당 이점을 가장 잘 활용할 수 있는 방식으로 결함 생성 작업을 지원하고 있다.

1.[이미지 생성 모델 이용하기] 코드 방문하기]

화면설계서1 중 이용하기와 GitHub 코드 방문하기 버튼에 대응 되는 버튼으로, 홈화면으로 되돌아가서 방문하는 번거로움을 줄였다.

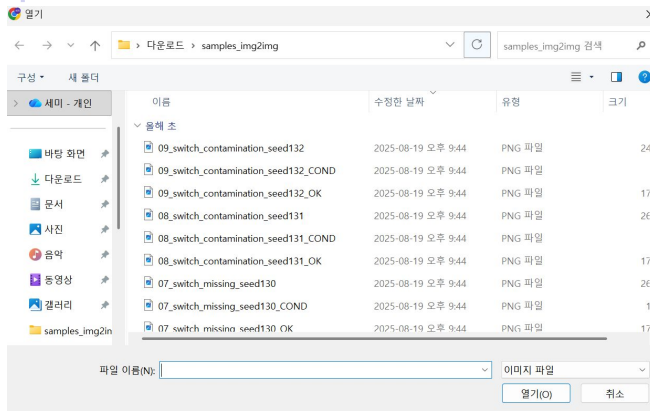
| UI/UX 정의서 - 화면 설계서 (3)

- 정상 이미지 입력 > 결함 MASK 이미지 입력 > 이미지 생성 후 반환 > 이미지 다운 및 평가지표 확인 가능

1. [이미지 입력창]

화면의 왼쪽에 위치한 OK이미지입력창과 controlNet 마스크 조건 이미지 입력창을 각각 클릭하면 자동으로 파일 선택 팝업창이 켜지면서 이미지 선택이 가능하다.
(하단의 이미지 선택 팝업창 참고.)

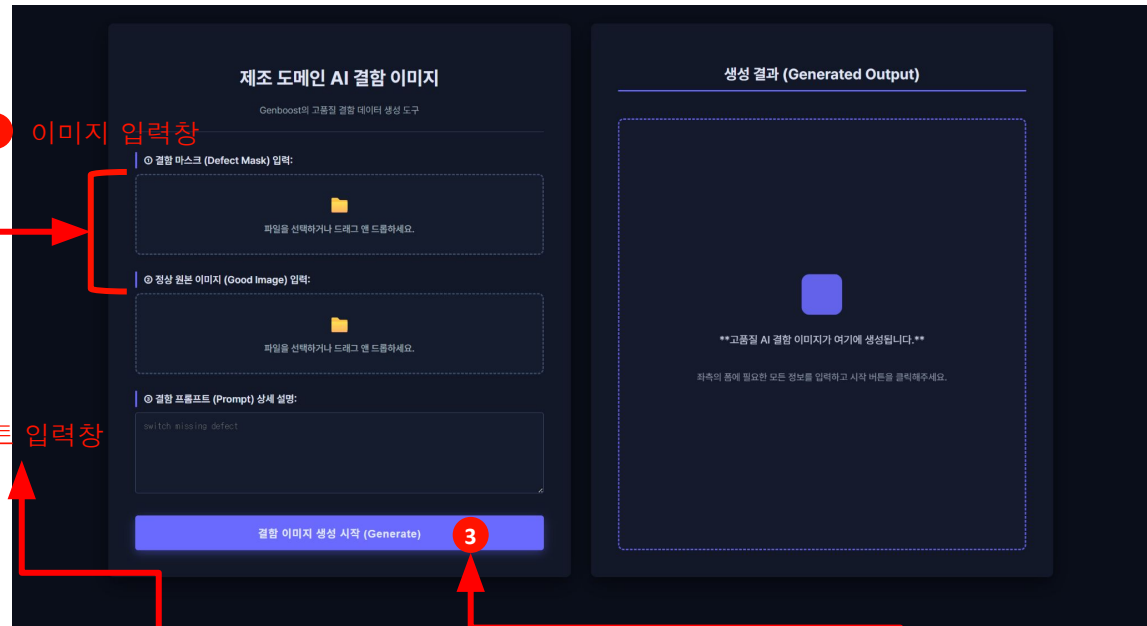
1-1. [이미지 입력창 (팝업창)]



1 이미지 입력창

2 텍스트 입력창

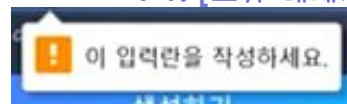
3



2. [프롬프트 입력]

모델에게 결함의 종류와 위치, 또는 제작하고자 하는 사물의 특징을 입력한다. 만약 미입력시 아래와 같은 오류 창을 띄운다.

1-1. [오류 메시지]

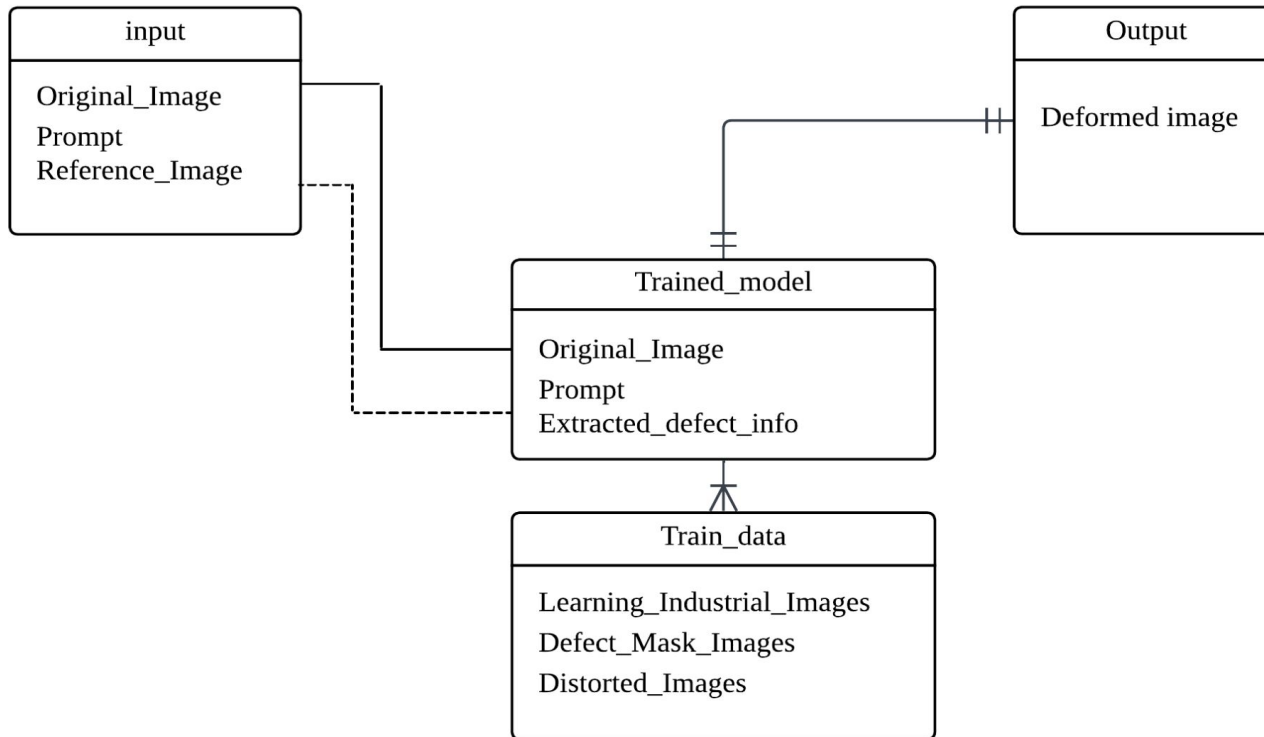


3. [생성하기]

생성 버튼을 누르면 입력한 이미지가 학습된 모델로 전송되며 서버에서 결함 이미지를 생성한다. 베이스 서버에 저장 후 다음 화면을 통해 생성된 이미지의 미리보기와 평가지표를 함께 출력하여 사용자에게 제공한다.

| 엔티티 관계도 - ERD

➤ 결함 이미지 생성 인공지능의 ERD(Entity-Relationship Diagram)

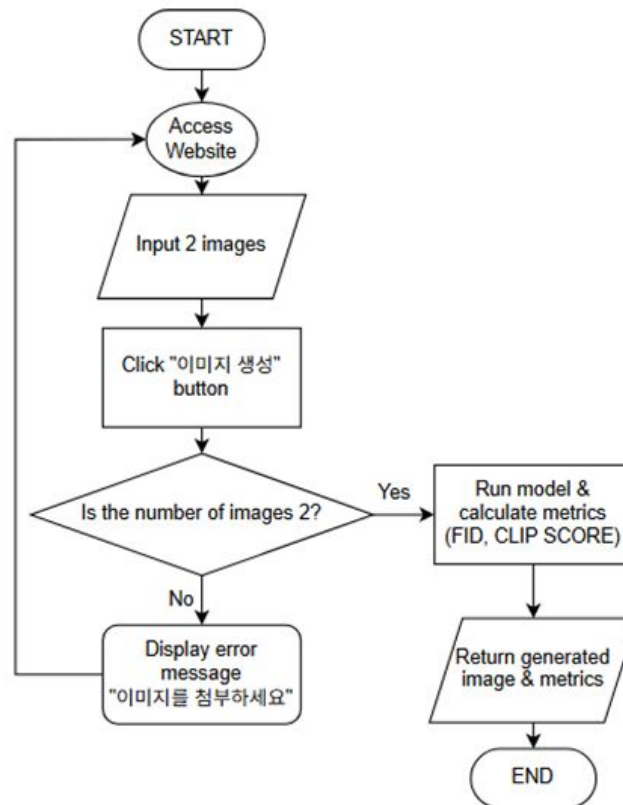


| 기능 처리도(기능 흐름도)

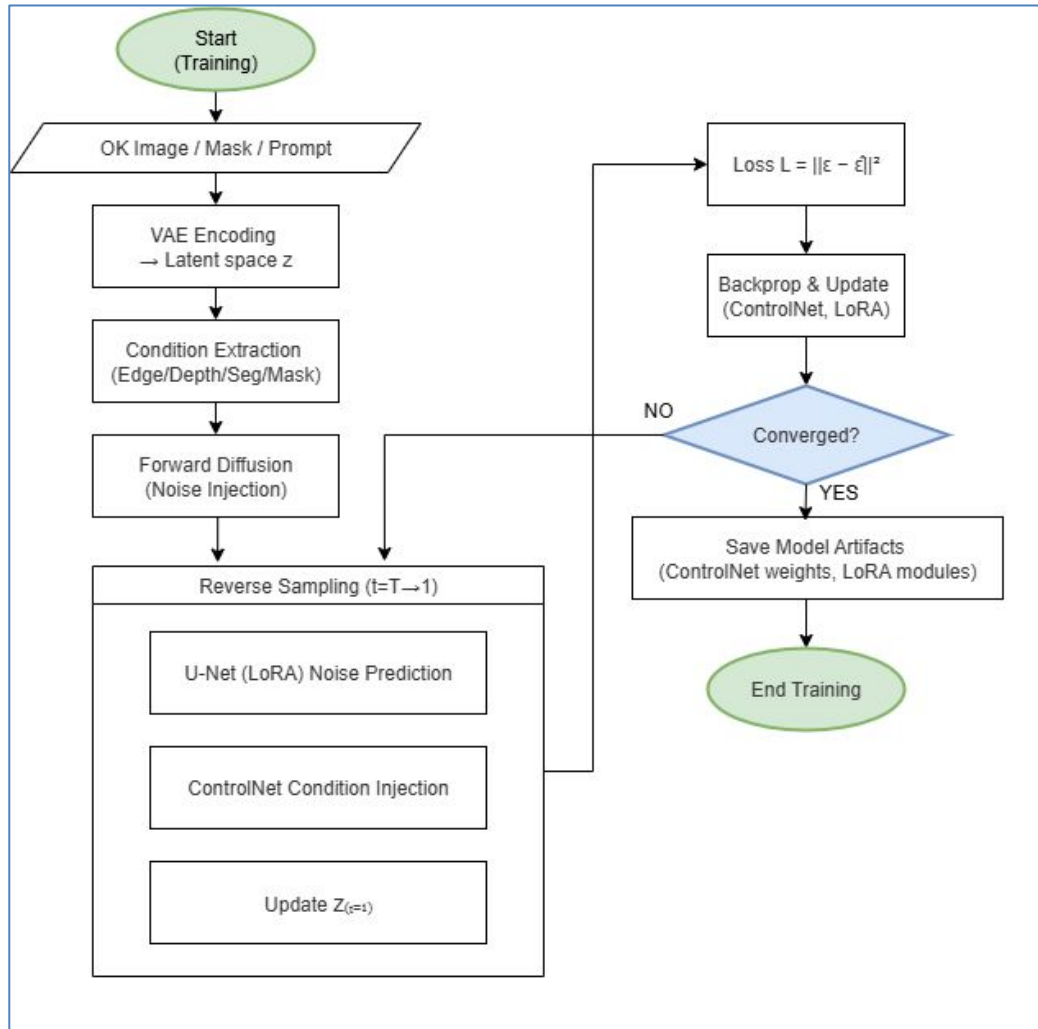
실무 산출물 형식

프로그램 ID		프로그램 명	결합 이미지 생성 인공지능	작성일	2025.08.22 .	Page	1/1
개요	사용자의 입력 이미지를 기반으로 새로운 결합 이미지를 생성하며, 그 결과에 대한 평가 지표(LPIPS)를 생성한 이미지와 함께 사용자에게 제공한다.					작성자	

기능 흐름도



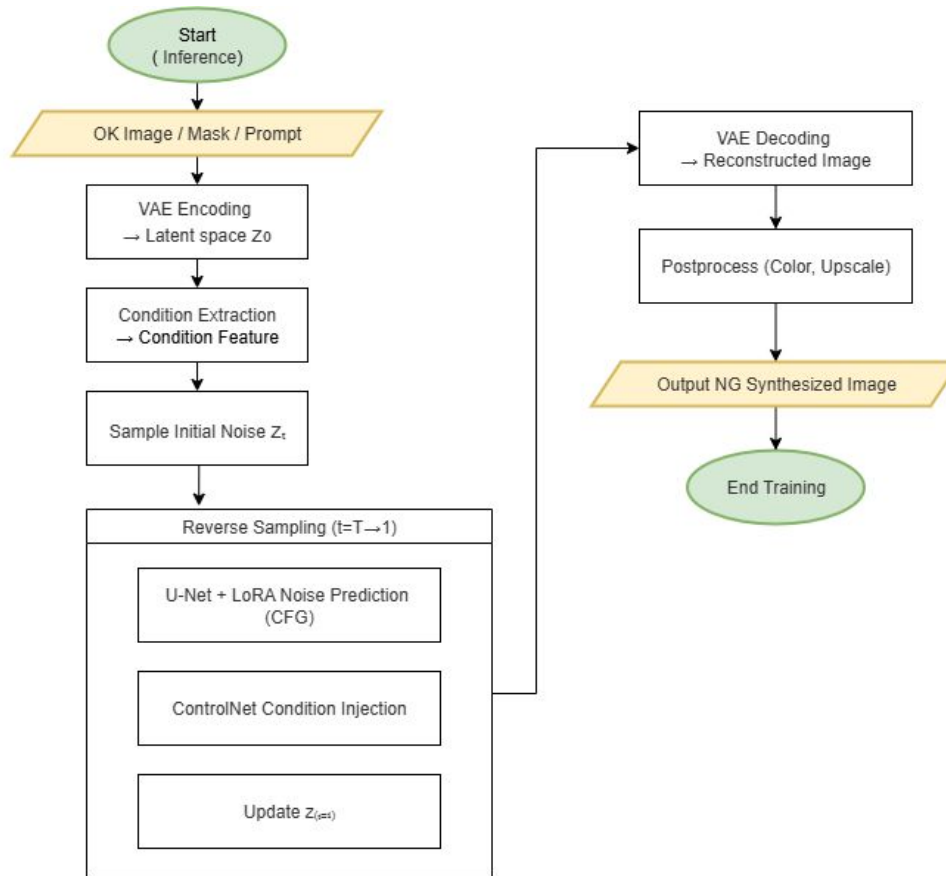
| 알고리즘 명세서 (1)



Train 알고리즘 흐름도

1. OK 이미지, NG 마스크, 텍스트 조건 데이터를 학습 입력으로 준비한다.
2. VAE Encoder를 통해 이미지를 잠재 공간(latent space)으로 변환한다.
3. 조건 입력(결합 마스크, Edge, Depth 등)을 Condition Encoder로 처리해 multi-scale feature로 변환한다.
4. Forward Process에서 원본 잠재 벡터에 점진적으로 노이즈를 주입한다.
5. Reverse Process 단계에서 U-Net(LoRA 적용)과 ControlNet이 협력하여 노이즈를 제거하고 조건 반영을 학습한다.
예측된 노이즈와 실제 노이즈 차이를 기반으로 손실 (Loss)을 계산한다.
6. 역전파를 통해 ControlNet 파라미터와 U-Net LoRA 모듈만 업데이트한다.
학습이 수렴할 때까지 4~7 과정을 반복한다.
7. 학습이 완료되면 ControlNet 가중치와 LoRA 모듈을 저장한다.

| 알고리즘 명세서 (2)



Inference 알고리즘 흐름도

1. 사용자가 OK 이미지, 결함 마스크, 텍스트 조건 (클래스명/결함명)을 입력한다.
2. 입력된 OK 이미지를 VAE Encoder로 변환하여 잠재 공간(latent space) 표현을 얻는다.
3. 결함 마스크는 Condition Encoder에서 multi-scale feature로 변환된다.
4. 초기 노이즈가 잠재 공간에 샘플링되어 결함 합성 과정의 시작점으로 사용된다.
5. Reverse Sampling 단계에서 U-Net(LoRA 적용)이 노이즈를 예측하고, ControlNet이 조건 feature를 주입한다.
6. 단계별로 노이즈가 제거되면서 결함이 반영된 잠재 벡터가 점차 복원된다.
7. VAE Decoder를 통해 잠재 벡터를 실제 이미지로 변환한다.
8. 색상 보정, 업스케일 등 후처리를 적용한다.
9. 최종적으로 합성된 NG 이미지를 출력한다.

| 알고리즘 상세 설명서(1)

Denoising Diffusion Probabilistic Model (DDPM)

고해상도 이미지를 직접 생성하기 어려운 문제를 해결하기 위해, 확률적 노이즈 추가 및 제거 과정을 통해 이미지를 생성하는 방식이다.

- 정방향 과정 (Forward Process)
깨끗한 이미지에 점진적으로 노이즈를 추가하여 완전한 가우시안 노이즈로 변환하는 과정이다.
- 역방향 과정 (Reverse Process)
학습된 신경망, 일반적으로 U-Net 기반 모델을 통해 노이즈를 제거하며 원본 이미지를 복원하는 과정이다.

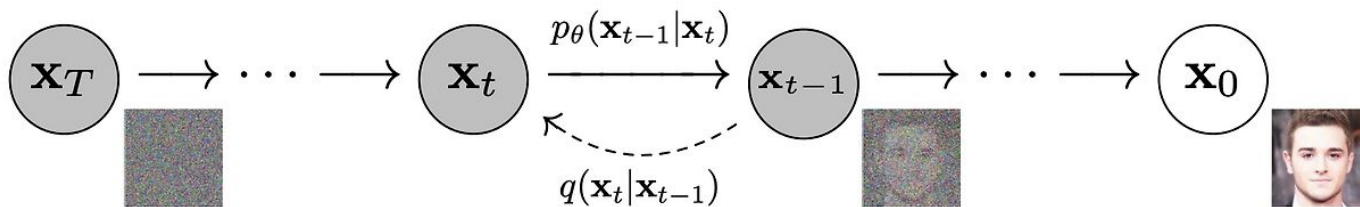


Figure 2: The directed graphical model considered in this work.

| 알고리즘 상세 설명서(2)

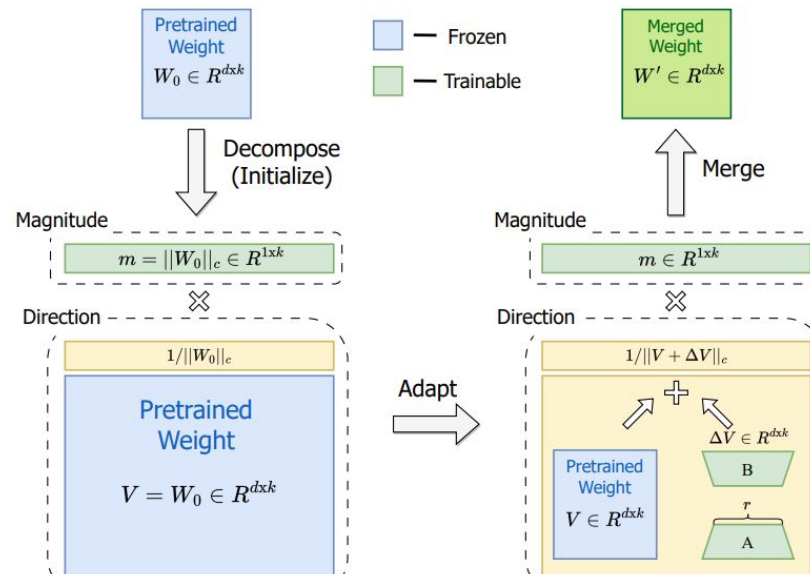
LoRA 기반 Fine-Tuning

A. 이론 배경:

- 모델 전체를 학습하지 않고, 특정 layer(weight matrix)에 대해 저차원 weight를 병렬 삽입하여 fine-tuning을 수행한다.
- 학습 파라미터 수 감소하기에 GPU 메모리 절약, 빠른 학습이 가능하다.

B. 적용 목적:

- 제조 도메인의 특화된 특성을 반영하기 위함이다.
- 불량의 미묘한 특징(스크래치, 눌림, 벗겨짐 등)을 학습하기 위한 소규모 fine-tuning을 하기 위함이다.



| 알고리즘 상세 설명서(3)

ControlNet (조건 기반 생성)

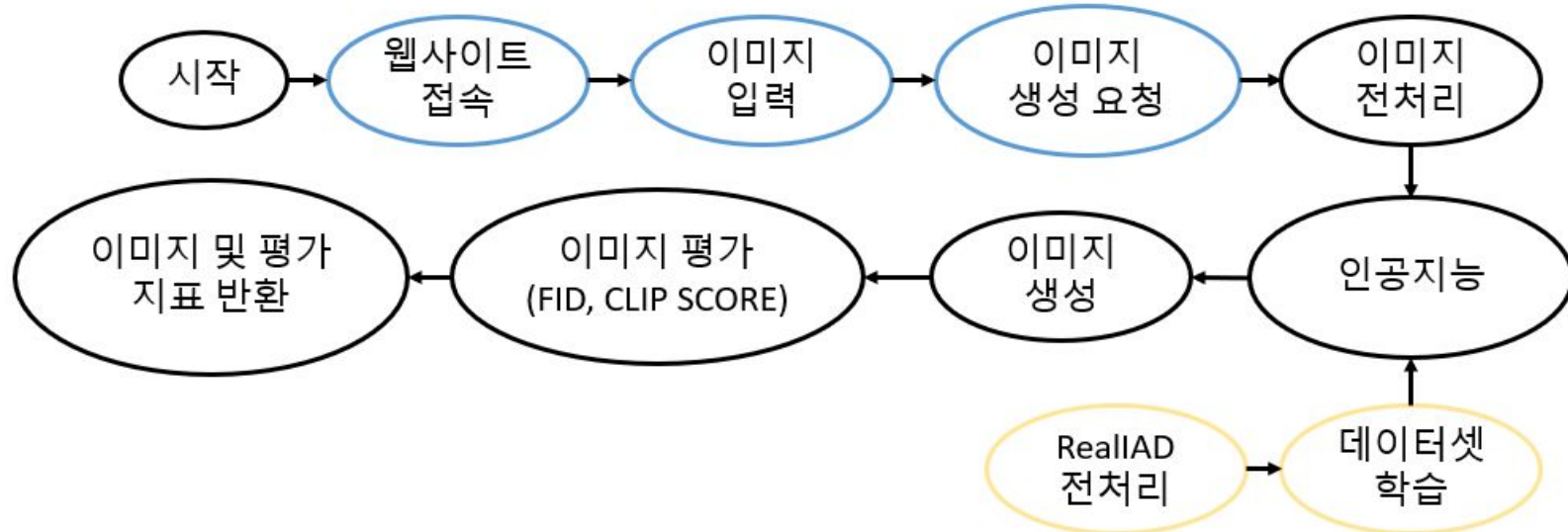
- 기반 이론
- A. 조건부 생성을 위해 외부 정보(예: edge map, pose, mask)를 보조 네트워크(ControlNet branch)로 처리하고, 이를 확산 모델의 UNet feature와 residual 연결하여 조건을 반영한다.
- B. Zero-initialization과 UNet 동결 방식을 통해 조건 반영력은 강화하면서도 pretrained 모델의 생성 능력은 유지한다.

- 동작 원리
 - a. 조건 입력 처리
 - i. 외부 조건을 ControlNet branch에 입력한다.
 - ii. Encoder/Decoder 레이어에서 특징(feature)을 추출한다.
 - b. Feature 결합
 - i. **ControlNet branch**에서 추출한 조건(feature)을 **UNet**의 대응되는 레이어와 **Residual** 연결한다.
 - ii. **UNet weight**는 동결(**freeze**)함으로써 사전 학습된 모델의 이미지 생성 능력은 유지된다.
 - c. 노이즈 제거(Denoising)
 - i. 입력 latent에 대해 반복적으로 노이즈를 제거한다.
 - ii. 조건 feature와 텍스트 prompt를 동시에 반영한다.
 - iii. 시간 단계마다 조건 정보를 반영하여 최종 이미지 생성한다.
 - d. 조건 반영 강도 조절
 - i. ControlNet weight 조절이 가능하기 때문에 조건 반영을 강하게 또는 약하게 제어할 수 있다.

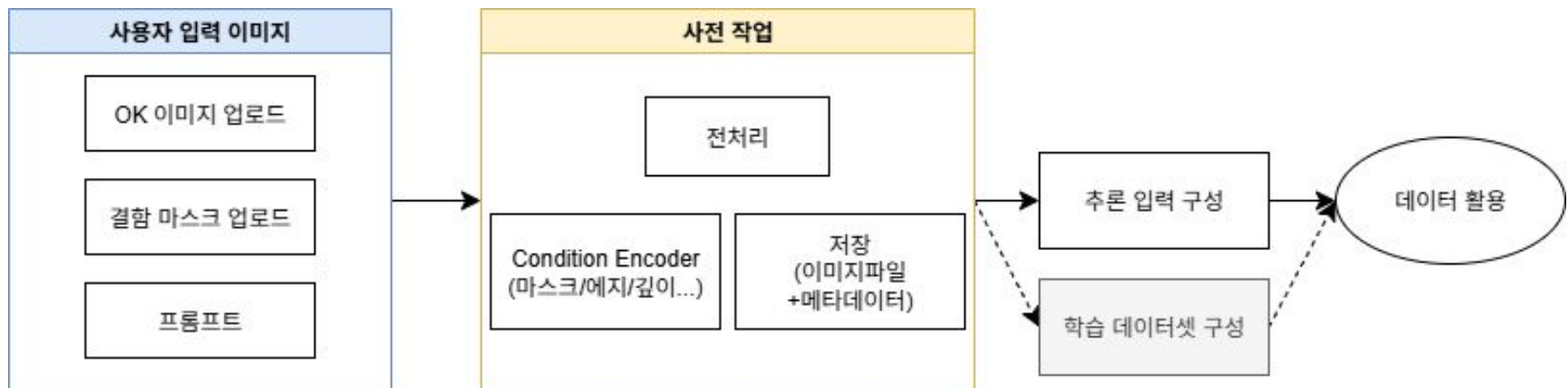
| 데이터 수집처리 정의서

사용자 입력 : 파란색 표시

사전 작업: 노란색 표시



* 웹에 업로드 된 데이터의 세부내용



| 프로그램 - 목록

기능 분류	기능 번호	기능명
Main	main-01	메인 화면
	main-01-01	GitHub 접속
	main-01-01-01	GitHub 주소
	main-01-02	프로젝트 소개
	main-01-03	이용하기
Use	use-01	OK 이미지 입력
	use-02	결합참고용 이미지 입력
	use-03	프롬프트 입력창
	use-04	결합 이미지 생성
Popup	pop-01	이미지 선택 팝업창 (OK 이미지)
	pop-02	이미지 선택 팝업창 (결합 이미지)
	pop-03	입력 실패 시 안내 메시지
	pop-03	이미지 생성 대기화면
Out	out-01	이미지 미리보기
	out-02	이미지 다운로드
	out-03	평가지표 출력 (CLIP, FID)
이하생략		

| 테이블 정의서 – ERD

범주	테이블	인덱스(컬럼)	용도
기본키/유니크	모든 테이블	id (PK) 자동, 유니크 제약된 컬럼	개별 행 식별, 무결성
이미지 관리	image_asset	sha256 (UNIQUE)	이미지 중복 방지
	image_asset	project_id	프로젝트별 이미지 검색
데이터셋/샘플	dataset	(project_id, name)	프로젝트 내 데이터셋 유니크
	sample	dataset_id, original_image_id, reference_image_id, prompt_id	샘플 → 데이터셋/이미지/프롬프트 연결
조건/어노테이션	condition_feature	(sample_id, type)	샘플별 조건(feature) 빠른 조회
	annotation	sample_id, defect_type_id	샘플별 결함 검색
모델 관리	model	(project_id, name)	프로젝트 내 모델명 유니크
	model_version	model_id	모델별 버전 조회
	model_artifact	(model_version_id, type)	버전별 아티팩트 검색
학습/추론	training_run	model_version_id, dataset_id	어떤 모델 버전+데이터셋으로 학습했는지
	inference_job	model_version_id, created_at, original_image_id	모델 버전/기간/원본별 추론 검색
	inference_output	job_id	추론 결과 추적
서치	JSONB 컬럼들(meta, parameters, metrics)	GIN 인덱스	조건 기반 필터링

| 핵심 소스코드(1)

모델 구조 파이프라인 Code

```
# ----- MODELS -----

vae = AutoencoderKL.from_pretrained(
    VAE_ID, subfolder="vae", torch_dtype=torch.float16
).to(device).eval()
for p in vae.parameters(): p.requires_grad = False

tokenizer = CLIPTokenizer.from_pretrained("openai/clip-vit-large-patch14")
text_encoder = CLIPTextModel.from_pretrained(
    "openai/clip-vit-large-patch14", torch_dtype=torch.float16
).to(device).eval()
for p in text_encoder.parameters(): p.requires_grad = False

def find_lora_targets(model):
    leaf_types = (torch.nn.Linear,
                  torch.nn.Conv1d, torch.nn.Conv2d, torch.nn.Conv3d)
    keep = set()
    for full_name, module in model.named_modules():
        if isinstance(module, leaf_types):
            last = full_name.split(".")[1]
            if not last.isdigit():
                keep.add(last)
    return sorted(keep)

# 1) ControlNet (PEFT-LoRA)
controlnet = ControlNetModel.from_pretrained(
    CONTROLNET_ID, torch_dtype=torch.float16
)

target_mods_ctrl = find_lora_targets(controlnet)
print("ControlNet LoRA 대상(샘플):", target_mods_ctrl[:20], "...")

lora_cfg_ctrl = LoraConfig(r=8, lora_alpha=16, bias="none",
                           target_modules=target_mods_ctrl)
controlnet = get_peft_model(controlnet, lora_cfg_ctrl).to(device)
controlnet.print_trainable_parameters()
controlnet.train()
```

```
# 2) 파이프라인 구성
pipe = StableDiffusionControlNetPipeline.from_pretrained(
    SD_ID, controlnet=controlnet, vae=vae,
    text_encoder=text_encoder, tokenizer=tokenizer,
    torch_dtype=torch.float16,
).to(device)

# 메모리 최적화
try: pipe.enable_xformers_memory_efficient_attention()
except Exception: pass
pipe.unet.to(memory_format=torch.channels_last)
pipe.controlnet.enable_gradient_checkpointing()
# 필요 시: pipe.unet.enable_gradient_checkpointing()

# 3) 기본 requires_grad
for p in pipe.unet.parameters(): p.requires_grad = False
for p in pipe.vae.parameters(): p.requires_grad = False
for p in pipe.text_encoder.parameters(): p.requires_grad = False
for p in pipe.controlnet.parameters(): p.requires_grad = True

# 4) UNet에 PEFT-LoRA 주입 (어텐션 루팅층)
lora_cfg_unet = LoraConfig(
    r=UNET_LORA_RANK,
    lora_alpha=UNET_LORA_RANK,
    target_modules=["to_q", "to_k", "to_v", "to_out.0"],
    bias="none",
)
pipe.unet = get_peft_model(pipe.unet, lora_cfg_unet)

# down 일부 동결(예: 초기 저주파 특성 과도학습 방지) - 필요 시 완화
for name, param in pipe.unet.named_parameters():
    if "lora_" in name and name.startswith("down_blocks"):
        param.requires_grad = False

tn_unet = sum(p.numel() for p in pipe.unet.parameters() if p.requires_grad)
tn_ctrl = sum(p.numel() for p in pipe.controlnet.parameters() if p.requires_grad)
print(f"Trainable params -> UNet(LoRA): {tn_unet:,} | ControlNet(LoRA): {tn_ctrl:,}")
pipe.unet.train()
```


| 핵심 소스코드(2)

학습 파이프라인 Code

```
# ----- TRAIN LOOP -----
global_step = 0
for epoch in range(1, EPOCHS + 1):
    total_loss = 0.0

    for batch in tqdm(loader, desc=f"Epoch {epoch}/{EPOCHS}"):
        # @ 배치 언팩
        lat_ok = batch["ok_lat"].to(device, torch.float16, non_blocking=True) # [B,4,64,64]
        lat_ng = batch["ng_lat"].to(device, torch.float16, non_blocking=True) # [B,4,64,64]
        cond = batch["cond"].to(device, torch.float16, non_blocking=True) # [B,3,512,512] in [-1,1]
        class_ids = batch["class_id"]
        defect_ids = batch["defect_id"]
        bsz = lat_ok.size(0)

        # @ 노이즈 & 타임스텝 (★ 교정: NG 기준, ε/velocity 예측)
        target_lat = lat_ng
        noise = torch.randn_like(target_lat)
        timesteps = torch.randint(0, noise_scheduler.config.num_train_timesteps,
                                   (bsz,), device=device)
        noisy_lat = noise_scheduler.add_noise(target_lat, noise, timesteps)

        # @ 텍스트 임베딩(캐시)
        with torch.no_grad():
            text_emb = get_text_embeds(class_ids, defect_ids) # (B,77,768)

        # @ forward → ControlNet + UNet
        optimizer.zero_grad(set_to_none=True)

        ctrl_out = pipe.controlnet(
            sample=noisy_lat,
            timestep=timesteps,
            encoder_hidden_states=text_emb,
            controlnet_cond=cond, # [B,3,512,512] in [-1,1]
            return_dict=True,
        )
        lat_pred = pipe.unet(
            sample=noisy_lat,
            timestep=timesteps,
            encoder_hidden_states=text_emb,
            down_block_additional_residuals=ctrl_out.down_block_res_samples,
            mid_block_additional_residual=ctrl_out.mid_block_res_sample,
            return_dict=True,
        ).sample # 예측값
```

```
# @ 손실 (ε-예측 기준 / v-예측 자동 대응)
if pred_type == "v_prediction":
    target = noise_scheduler.get_velocity(target_lat, noise, timesteps)
else:
    target = noise # epsilon

loss = F.mse_loss(lat_pred.float(), target.float(), reduction="none")
loss = loss.mean(dim=(1,2,3)) # per-sample

# (선택) SNR 가중
if USE_SNR_WEIGHT:
    snr = get_snr(timesteps).to(loss.dtype) # (B,)
    # Imagen에서 제안한 gamma-weighted loss (가벼운 변형)
    weight = min(snr, gamma) / snr # 저주파/고주파 균형
    weights = (snr.clamp(max=SNR_GAMMA) / snr).detach()
    loss = loss * weights

loss = loss.mean()
loss.backward()

# 그래디언트 클립 & 스텝
torch.nn.utils.clip_grad_norm_(params_controlnet + params_unet_lora, GRAD_CLIP_NORM)
optimizer.step()
scheduler.step()

# @ 로그 & 체크포인트
total_loss += float(loss.item())
global_step += 1

if global_step % 100 == 0:
    lr1 = scheduler.get_last_lr()[0]
    lr2 = scheduler.get_last_lr()[-1]
    print(f"[step {global_step}] loss={loss.item():.4f} | lr_ctrl={lr1:.2e} lr_unet={lr2:.2e}")

if global_step % SAVE_STEPS == 0:
    # ControlNet(LoRA) 저장
    save_dir_c = f"{OUTPUT_DIR}/ctrl_lora_step{global_step}"
    controlnet.save_pretrained(save_dir_c)
    # UNet(LoRA) 저장
    save_dir_u = f"{OUTPUT_DIR}/unet_lora_step{global_step}"
    pipe.unet.save_pretrained(save_dir_u)
    print(f"[ckpt] saved: {save_dir_c} | {save_dir_u}")

avg = total_loss / max(1, len(loader))
print(f"Epoch {epoch} finished | avg loss: {avg:.4f}")

# 에폭 소냅샷
controlnet.save_pretrained(f"{OUTPUT_DIR}/ctrl_lora_epoch{epoch}")
pipe.unet.save_pretrained(f"{OUTPUT_DIR}/unet_lora_epoch{epoch}")
print(f"[ckpt] epoch {epoch} saved.")
```


| 핵심 소스코드 (3)

추론 파이프라인 Code

```
# ----- Pipeline Loader -----

def load_pipe_full(sd_id, controlnet_id, unet_lora, ctrl_lora, vae_id, fp16, use_xformers, mode):
    torch_dtype = torch.float16 if (fp16 and torch.cuda.is_available()) else torch.float32

    base_unet = UNet2DConditionModel.from_pretrained(sd_id, subfolder="unet", torch_dtype=torch_dtype)
    base_controlnet = ControlNetModel.from_pretrained(controlnet_id, torch_dtype=torch_dtype)

    unet = merge_module(base_unet, unet_lora, "UNet")
    controlnet = merge_module(base_controlnet, ctrl_lora, "ControlNet")

    if mode == "inpaint":
        PipeCls = StableDiffusionControlNetInpaintPipeline
    elif mode == "img2img":
        PipeCls = StableDiffusionControlNetImg2ImgPipeline
    else:
        PipeCls = StableDiffusionControlNetPipeline

    kwargs = dict(unet=unet, controlnet=controlnet, torch_dtype=torch_dtype, safety_checker=None)
    if vae_id:
        vae = AutoencoderKL.from_pretrained(vae_id, subfolder="vae", torch_dtype=torch_dtype)
        pipe = PipeCls.from_pretrained(sd_id, vae=vae, **kwargs)
    else:
        pipe = PipeCls.from_pretrained(sd_id, **kwargs)

    pipe.scheduler = DPMSolverMultistepScheduler.from_config(pipe.scheduler.config)
    if use_xformers:
        try: pipe.enable_xformers_memory_efficient_attention()
        except: pass

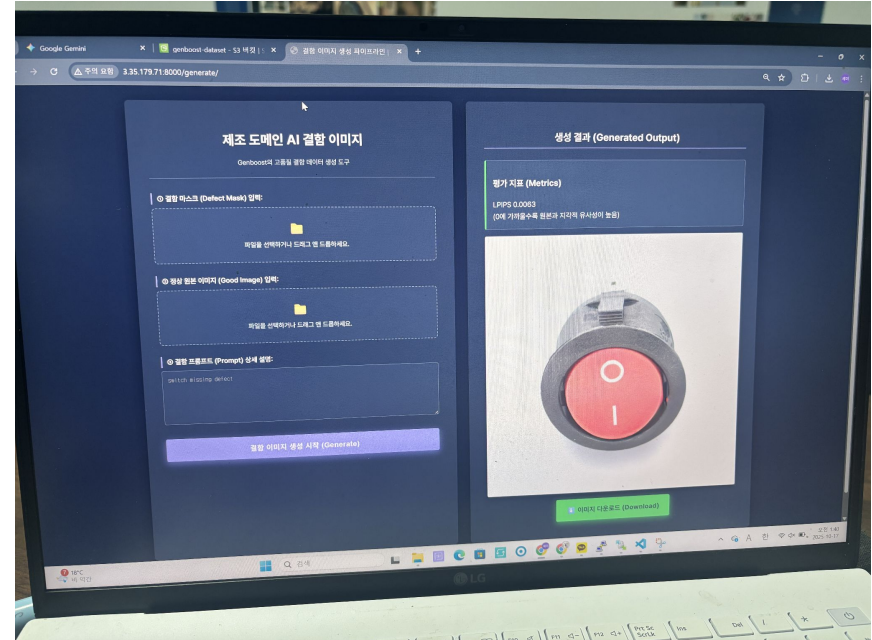
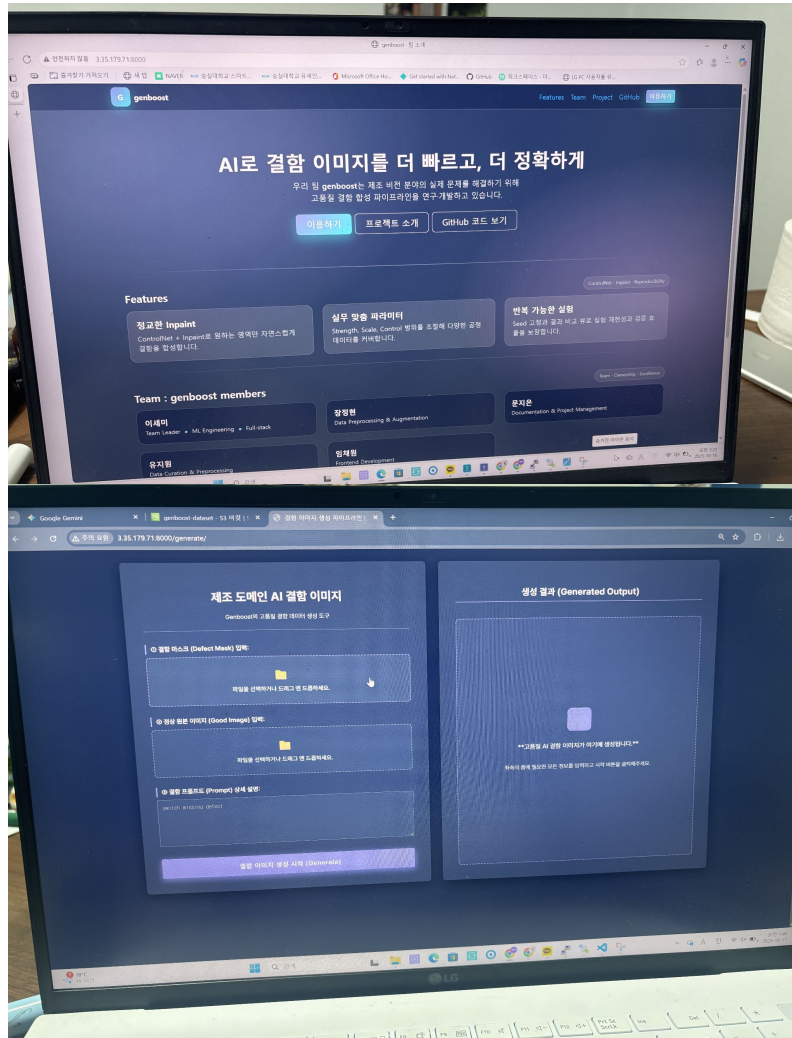
    pipe.unet.to(memory_format=torch.channels_last)
    pipe.controlnet.to(memory_format=torch.channels_last)
    pipe.to("cuda" if torch.cuda.is_available() else "cpu")
    return pipe
```

```
for i in range(args.n):
    g = gen.manual_seed(args.seed + i)

    if use_inpaint:
        # Inpaint + ControlNet
        result = StableDiffusionControlNetInpaintPipeline._call_(
            pipe,
            prompt=args.prompt,
            negative_prompt=neg,
            image=init_rgb,
            mask_image=mask,
            control_image=cond_rgb,
            num_inference_steps=args.steps,
            guidance_scale=args.scale,
            strength=float(min(max(args.strength, 0.0), 1.0)),
            controlnet_conditioning_scale=args.cscale,
            control_guidance_start=args.cstart,
            control_guidance_end=args.cend,
            generator=g,
        )
    else:
        # text2img + ControlNet
        result = StableDiffusionControlNetPipeline._call_(
            pipe,
            prompt=args.prompt,
            negative_prompt=neg,
            image=cond_rgb,
            num_inference_steps=args.steps,
            guidance_scale=args.scale,
            controlnet_conditioning_scale=args.cscale,
            control_guidance_start=args.cstart,
            control_guidance_end=args.cend,
            generator=g,
        )

    out = result.images[0]
    fname = f"{i:02d}_{base_name}_seed{args.seed+i}.png"
    out.save(os.path.join(args.out, fname))
    print("[saved]", fname)
```

| 참조-S/W 기능 실사 사진



실제 여러 노트북이나 컴퓨터로
웹페이지를 정상적으로 접속하여
원활하게 작동됨을 확인할 수 있음

| 참조- 개발 환경 및 서면

구분		상세내용	역할
S/W 개발환경	OS	Ubuntu 22.04 LTS	
	개발환경(IDE)	Visual Studio Code, Google Colab	
	개발도구	Git, Jupyter Notebook, Conda	
	개발언어	Python	모델 학습 및 파인 튜닝에 사용할 언어
	모델 아키텍처	Stable Diffusion v1.5	생성형 모델
	프론트엔드	HTML, CSS, JavaScript	결과 시각화, 사용자 입력
	기타사항	CUDA, Hugging Face (Transformers, Diffusers), AI/ML 프레임워크 및 라이브러리(PyTorch, OpenCV, Scikit-learn 등)	모델 사용에 필요한 베이스 구축
프로젝트 관리환경	형상관리	GitHub	코드 관리
	실험 관리	GitHub, Jupyter Notebook	실험 코드 및 결과 관리
	배포 환경	AWS EC2 (g5.2xlarge / A10G GPU), S3, EBS	GPU 학습, 결과 저장

| 참조-프로젝트 관리

프로젝트 관리 -Zoom, 카카오톡, Notion 이용하여 팀원들과 소통

Diffusion Models 후보군	2025년 7월 4일 오후 10:20	● 시작 전
AWS 접속 참고용	2025년 7월 20일 오후 12:08	● 시작 전
제출용 보고서 ☐ 열기	2025년 7월 9일 오후 8:30	● 진행 중
이미지 전처리 코드 ver 1.	2025년 7월 23일 오후 9:35	● 시작 전
이미지 전처리 코드 ver2.	2025년 7월 31일 오전 3:09	● 시작 전
이미지 전처리 코드 ver3.	2025년 8월 5일 오전 11:07	● 진행 중
T2I-Adapter 모델 성능 평가	2025년 8월 3일 오전 2:16	● 완료
Controlnet++ 모델 성능 평가	2025년 8월 6일 오후 7:43	● 시작 전

공유된 페이지

 문서 허브

 회의록

 프로젝트

7월 멘티 줌 미팅	2025년 7월 4일
7월 멘토&멘티 줌 미팅	2025년 7월 6일
7월 20일 오프라인 미팅 ☐ 열기	2025년 7월 20일
8월 20일 오프라인 미팅까지 할 일	2025년 7월 23일
7월 오프라인 미팅	2025년 7월 23일
8월 3일 줌 회의	2025년 8월 3일

Thank
you

